

هندسة البرمجيات 2

الوظيفة 2

تقديم: غنى فاروق أحمد

الفئة الخامسة

الطلب الأول: اختيار التابع ذو التعقيد المتوسط (Source Code Selection)

بناءً على المتطلبات الوظيفية المطروحة في وثيقة مواصفات النظام (SRS) لمشروع MediChain

وتحديداً نظام جدولة المواعيد بموديول الأشعة، تم تصميم وكتابة التابع التالي بلغة Java.

يعالج هذا التابع عملية "الحجز والجدولة التلقائية" لفحوصات الأشعة بناءً على أربعة محددات رئيسية مستخلصة من نص الزبون: (تعليمات التحضير، سعة الأجهزة والموارد، وجود تضارب في المواعيد، وأولوية الحالة الطبية للمريض). تم صياغة لضمان الحصول على قيمة تعقيد دوري المنطق البرمجي عمداً ليحتوي على شروط وتقرعات متداخلة (Nested conditions)

(CC) متوسطة وعالية تحقق المعيار المطلوب أكبر من 3

(Original Java Code) الكود البرمجي للتابع الأصلي:

```
public class RadiologyScheduler {

    /**
     * تابع الجدولة التلقائية لفحوصات الأشعة *
     */

    public String autoScheduleRadiology(int duration, boolean prepDefined, int capacity, String priority,
    boolean hasConflict) {

        // القيد الإلزامي الأول: التحقق من وجود وتحديد تعليمات التحضير الطبي للفحص 1.
        if (!prepDefined) {

            return "Rejected: Preparation instructions are required.";

        }

    }

}
```

// القيد الإلزامي الثاني: التحقق من الطاقة الاستيعابية وسعة جهاز الأشعة المطلوبة. 2 //

if (capacity <= 0) {

return "Reschedule: Device capacity exceeded.";

}

// معالجة حالات التضارب والتزام على نفس الجهاز والموعِد. 3 //

if (hasConflict) {

// تفريع بناءً على الأولوية والحالات الحرجة //

if (priority.equals("EMERGENCY")) {

return "Scheduled: High Priority Override.";

}

// تفريع لتحسين استغلال الوقت في المواعيد القصيرة //

else if (duration < 30) {

return "Scheduled: Optimized Short Duration Slot.";

}

// حالة التضارب العادية بدون ميزات تفضيلية //

else {

return "Queued: Waiting for next available slot.";

}

}

// المسار الافتراضي في حال عدم وجود أي تضارب أو عوائق. 4 //

return "Scheduled: Standard Slot Allocated.";

}

}

الطلب الثاني: حساب قيمة التعقيد الدوري يدوياً وتوثيقها (Cyclomatic Complexity - CC) :

تم حساب قيمة التعقيد الدوري عبر طريقتين علميتين مختلفتين لضمان التطابق الرياضي والبرمجي:

أولاً: بناء مخطط تدفق التحكم (Control Flow Graph – CFG):

نقوم أولاً بترقيم الأسطر البرمجية والتحويلات المنطقية داخل التابع وتحويلها إلى عُقد (Nodes) وحواف (Edges):

عقدة 1 (Node1): بداية التابع واستلام المقاييس ورأس التابع

عقدة 2 (Node2): الفحص الشرطي الإلزامي الأول (if (!prepDefined))

عقدة 3 (Node3): نقطة الخروج الأولى (إرجاع رفض لعدم وجود تعليمات التحضير)

عقدة 4 (Node4): الفحص الشرطي الإلزامي الثاني (if (capacity <= 0))

عقدة 5 (Node5): نقطة الخروج الثانية (إرجاع طلب إعادة جدولة بسبب السعة)

عقدة 6 (Node6): الفحص الشرطي الثالث المتعلق بوجود تضارب (if (hasConflict))

عقدة 7 (Node7): فحص الأولوية داخل بنية التضارب المتداخلة (if (priority.equals("EMERGENCY")))

عقدة 8 (Node8): نقطة الخروج الثالثة (تأكيد موعد طوارئ متجاوز للتضارب)

عقدة 9 (Node9): فحص المدة الزمنية للفحص (else if (duration < 30))

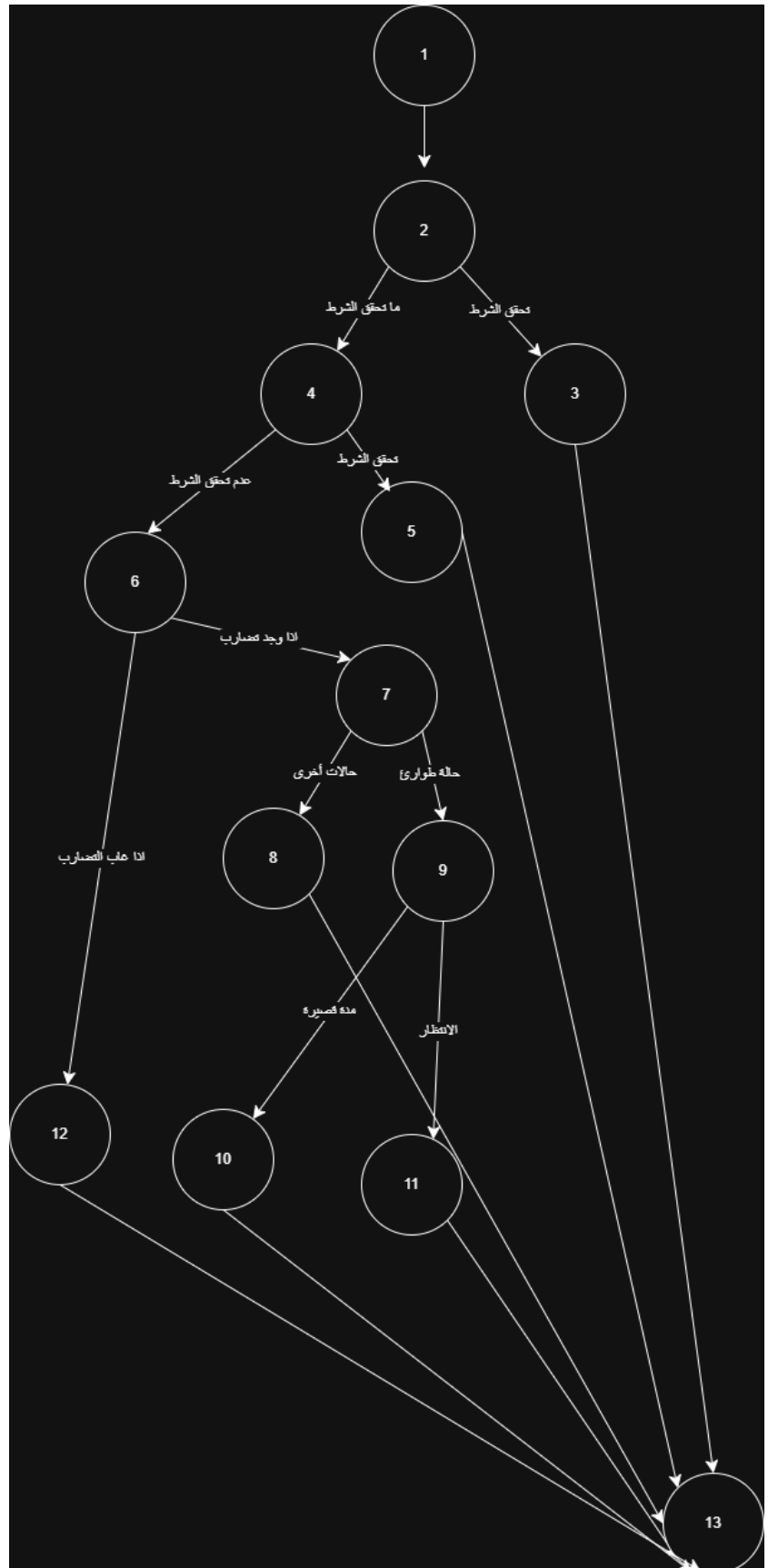
عقدة 10 (Node10): نقطة الخروج الرابعة (تخصيص موعد للمدد القصيرة المحسنة)

عقدة 11 (Node11): نقطة الخروج الخامسة (وضع الطلب في قائمة الانتظار لعدم توفر شروط التفضيل)

عقدة 12 (Node12): نقطة الخروج السادسة (تخصيص موعد قياسي عند غياب التضارب)

عقدة 13 (Node13): العقدة النهائية الموحدة لانتهاج التابع (Exit Node)

قمت برسمه للمخطط على draw.io:



ثانياً: الحساب الرياضي الاعتمادي (Analytical Calculation):

1. طريقة العقد الشرطية المتفرعة (Predicate Nodes Method):

تعتمد هذه الطريقة على حصر القرارات الشرطية التي يتفرع منها مساران مستقلان داخل الكود البرمجي

(Predicate Nodes) وترمز بـ P (الشروط المتفرعة في تابعا هي :

1 | الشرط المرتبط بالتحضير: (!prepDefined) if

2 | الشرط المرتبط بالسعة: (capacity <= 0) if

3 | الشرط المرتبط بالتضارب: (hasConflict) if

4 | الشرط المرتبط بالطوارئ: (priority.equals("EMERGENCY")) if

5 | الشرط المرتبط بمدة الجلسة: (duration < 30) else if

وبالتالي فإن عدد العقد الشرطية P=5

بتطبيق المعادلة الرسمية:

$$CC = P + 1 = 4 + 1 = 5$$

2 طريقة المناطق المغلقة (Regions Method):

عند تمثيل مخطط تدفق التحكم (CFG) بيانياً نلاحظ تقسم الرسم إلى مساحات محصورة ومغلقة تماماً ناتجة عن التفرعات والاتفاقات حول العقد، بالإضافة إلى الفضاء الخارجي المحيط بالرسم بالكامل كمنطقة مفتوحة أخيرة.

عدد المناطق المغلقة الناتجة عن التفرعات = 4 مناطق.

المنطقة الخارجية المفتوحة المحيطة بالمخطط = 1 منطقة.

المجموع الكلي للمناطق (Regions)=5.

تتطابق الطريقتان في إثبات أن قيمة التعقيد الدوري للتابع تساوي 5 وهي قيمة متوسطة ومثالية أكبر من 3.

الطلب الثالث: تصميم حالات الاختبار المناسبة (Test Cases Design) :

وفقاً لمنهجية اختبار البرمجيات على مستوى المسار الأساسي (Basis Path Testing) فإن قيمة التعقيد الدوري CC=5

شير خطياً وبشكل صارم إلى أن هذا التابع يحتوي على 5 مسارات مستقلة تماماً

اختبار مخصص وموجه لاختبار كل مسار على حدة:

(جدول توثيق حالات الاختبار) Test Cases Specification Table:

(ID) معرف الحالة	هدف الاختبار والمسار المستهدف	المدخلات الممررة للتابع (Inputs)	تعليل المسار وعمقه المنطقي
TC_01	اختبار قيد التحضير الطبي وغيابه المباشر	duration=40 prepDefined=false capacity=5 priority="ROUTINE" hasConflict=false	يختبر المسار الأول ويحقق الخروج المبكر فوراً عند أول جدار حماية للكود دون فحص بقية الشروط
TC_02	اختبار تخطي الشرط الأول والوقوف عند قيد السعة	duration=40 prepDefined=true	يختبر المسار الثاني، حيث يتجاوز الشرط الأول بنجاح، ويسقط عند شرط انعدام سعة الأجهزة

	capacity=0 priority="ROUTINE" hasConflict=false		
يختبر المسار الثالث المتقدم (وجود تضارب مع أولوية قصوى)، للتحقق من كفاءة النظام في تقديم الحالات الإسعافية	duration=40 prepDefined=true capacity=2 priority="EMERGENCY" hasConflict=true	اختبار ميزة تجاوز التضارب لحالات الطوارئ الحرجة	TC_03
يختبر المسار الرابع، تضارب موجود والأولوية عادية، لكن النظام يتخذ قراراً ذكياً بتمرير الفحص لأن مدته قصيرة جداً (< 30)	duration=20 prepDefined=true	اختبار استغلال التضارب عبر المدد القصيرة (Optimization)	TC_04

	capacity=2 priority="ROUTINE" hasConflict=true		
يختبر المسار الخامس والأخير، وهو المسار المثالي الذي يمر (Happy Path) عبر كل الشروط بنجاح ويحجز موعداً طبيعياً	duration=45 prepDefined=true capacity=2 priority="ROUTINE" hasConflict=false	اختبار الوصول إلى الجدولة القياسية بدون عوائق	TC_05

الطلب الرابع: تنفيذ الاختبار البرمجي وإرفاق الإثباتات (Test Execution & Execution Screenshot) :

لتحقيق أقصى درجات الوثوقية التطبيقية، تم بناء بيئة فحص برمجية متكاملة داخل بيئة التطوير NetBeans IDE عبر كتابة برنامج تنفيذي Console Application يقوم بتمرير حالات الاختبار الخمسة المصممة في الجدول أعلاه تلقائياً، وطباعة المخرجات الفعلية لمقارنتها بالمخرجات المتوقعة

:Testing Wrapper Code

```
public class RadiologySchedulerTest {

    public static void main(String[] args) {
        RadiologyScheduler scheduler = new RadiologyScheduler();

        System.out.println("=====");
        System.out.println("=== Running MediChain Radiology Module Tests ===");
        System.out.println("=====\\n");

        // TC_01: يجب رفض الطلب -> غياب تعليمات التحضير
        String res1 = scheduler.autoScheduleRadiology(40, false, 5, "ROUTINE", false);
        String exp1 = "Rejected: Preparation instructions are required.";
        System.out.println("[TC_01] Result : " + res1);
        System.out.println("[TC_01] Expected : " + exp1);
        System.out.println("[TC_01] Status : " + (res1.equals(exp1) ? "PASS ✓" : "FAIL ✗") + "\\n");

        // TC_02: إعادة جدولة -> سعة الجهاز = 0
        String res2 = scheduler.autoScheduleRadiology(40, true, 0, "ROUTINE", false);
        String exp2 = "Reschedule: Device capacity exceeded.";
        System.out.println("[TC_02] Result : " + res2);
        System.out.println("[TC_02] Expected : " + exp2);
        System.out.println("[TC_02] Status : " + (res2.equals(exp2) ? "PASS ✓" : "FAIL ✗") + "\\n");

        // TC_03: تجاوز التضارب -> تضارب + أولوية طوارئ
        String res3 = scheduler.autoScheduleRadiology(40, true, 2, "EMERGENCY", true);
        String exp3 = "Scheduled: High Priority Override.";
        System.out.println("[TC_03] Result : " + res3);
        System.out.println("[TC_03] Expected : " + exp3);
        System.out.println("[TC_03] Status : " + (res3.equals(exp3) ? "PASS ✓" : "FAIL ✗") + "\\n");

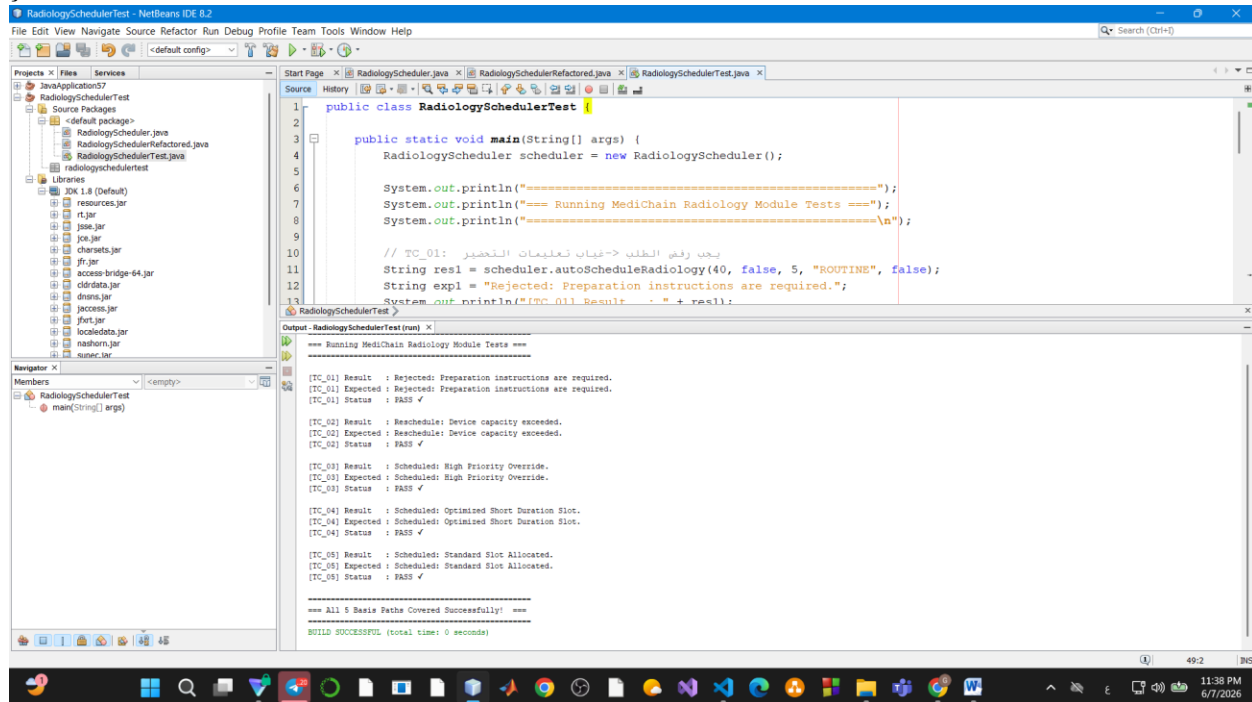
        // TC_04: موعد محسّن -> 30 < تضارب + مدة قصيرة
        String res4 = scheduler.autoScheduleRadiology(20, true, 2, "ROUTINE", true);
        String exp4 = "Scheduled: Optimized Short Duration Slot.";
        System.out.println("[TC_04] Result : " + res4);
        System.out.println("[TC_04] Expected : " + exp4);
        System.out.println("[TC_04] Status : " + (res4.equals(exp4) ? "PASS ✓" : "FAIL ✗") + "\\n");

        // TC_05: جدولة قياسية -> لا تضارب، جميع الشروط طبيعية
        String res5 = scheduler.autoScheduleRadiology(45, true, 2, "ROUTINE", false);
        String exp5 = "Scheduled: Standard Slot Allocated.";
        System.out.println("[TC_05] Result : " + res5);
        System.out.println("[TC_05] Expected : " + exp5);
        System.out.println("[TC_05] Status : " + (res5.equals(exp5) ? "PASS ✓" : "FAIL ✗") + "\\n");
    }
}
```

```

System.out.println("=====");
System.out.println("=== All 5 Basis Paths Covered Successfully! ===");
System.out.println("=====");
}
}

```



هذه صورة توثيق التنفيذ الناجح...

الطلب الخامس: إعادة الهيكلة البرمجية وتخفيض التعقيد الدوري (Refactoring)

تعتبر الأكواد ذات الشروط المتداخلة العميقة (Nested If-Else) بيئة خصبة لظهور الأخطاء البرمجية (Bugs) وصعبة الصيانة (Low Maintainability) لذلك، قمنا بإجراء عملية **Refactoring** احترافية للتابع البرمجي باستخدام تقنيات نمطية متقدمة في هندسة البرمجيات :

1 الاعتماد على الشروط الحارسة (Guard Clauses): معالجة القيود الحتمية والخروج المبكر للتخلص من الـ Nesting.

2 فصل التضارب خطياً (Linear Separation): فك التداخل البرمجي داخل شرط التضارب وجعله يتدفق عمودياً بشكل سلس وواضح للعين ومريح لذاكرة المعالجة.

(Refactored Code) لكود البرمجي بعد إعادة الهيكلة والتعديل:

```
public class RadiologySchedulerRefactored {

    /**
     * نسخة محسنة ومبسطة للتابع باستخدام الشروط الحارسة *
     */

    public String autoScheduleRadiologyRefactored(int duration, boolean prepDefined, int capacity, String
priority, boolean hasConflict) {

        // معالجة الاستثناءات والخروج الفوري - (Guard Clauses) الشروط الحارسة 1.
        if (!prepDefined) return "Rejected: Preparation instructions are required.";
        if (capacity <= 0) return "Reschedule: Device capacity exceeded.";

        // معالجة المسار الافتراضي المستقر عند غياب التضارب (توفير عمق تفريع كامل 2.
        if (!hasConflict) return "Scheduled: Standard Slot Allocated.";

        // بدلاً من البنية المتداخلة (Linear Evaluation) معالجة حالات التضارب بشكل خطي متتالي 3.
        if (priority.equals("EMERGENCY")) {
            return "Scheduled: High Priority Override.";
        }

        if (duration < 30) {
            return "Scheduled: Optimized Short Duration Slot.";
        }

        // المسار الأخير المتبقي تلقائياً
```

```
return "Queued: Waiting for next available slot.";  
  
}  
  
}
```

حساب قيمة التعقيد الدوري الجديد(CC)ومقارنته بالأصلي:

عند تحليل الكود الجديد بعد Refactoring: يتبين لنا ما يلي:

المنطق البرمجي أصبح خطياً تتابعياً(Straight-Line Coding) مما يعني إلغاء تداخل المسارات تماماً وسهولة تتبع الكود برمجياً وصيانتته مستقبلاً دون الخوف من تأثير الشروط الخارجية على التفرعات الداخلية. تخلصنا من البنية المتداخلة العميقة(Deep Nesting Level) وحولناها إلى مستويات مسطحة (Flat Level Structure) مما يقلل بشكل هائل من الجهد الإدراكي(Cognitive Complexit)المطلوب من أي مطور برمجيات آخر في فريق MediChain لفهم آلية عمل الجدولة، وهذا هو صلب وهدف علم هندسة البرمجيات المعاصرة